Dhirubhai Ambani University
Technology

Formerly DA-IICT

# WS & SOA (IT644)
# CodeTrail


| Name | Student ID |
|---|---|
| Monar | 202512019 |
| Rishika Shah | 202512041 |
| Meet Sheth | 202512057 |
| Ruchita Patadiya | 202512070 |
| Shaikh Maryam | 202512074 |
| Mahi Kansara | 202512078 |
| Anistina Dsouza | 202512081 |
| Thakkar Manushri | 202512122 |

# Project Timeline & Milestones

The creation of CodeTrail is based on an **iterative and incremental approach** which is inspired by **Agile** methods. Rather than carrying out all the development work first and then testing it, each module will be implemented, followed by unit testing, integration, code review, documentation and bug fixing, before the team moves on to the next module.

This policy of continuous testing **reduces the risks** associated with integration, enhances the quality of the software, and makes it possible to detect and remedy defects as early as possible in the development process.

The timeline set out below matches the academic deliverables and at the same time adheres to standard industry software engineering practices.

| **Month** | **Week** | **Milestone / Phase** | **Development Activities** | **Testing & Integration Activities** | **Expected Deliverable** |
|---|---|---|---|---|---|
| **July** | **Week 1** | Team Formation & Project Ideation | Team formation, literature survey, technology evaluation, feasibility analysis, and project idea finalization. | Review project feasibility and proposed solution. | Project group formation & Project idea submission |
| **August** | **Week 2** | Project Proposal | Finalize objectives, scope, technology stack, module allocation, responsibilities, and project roadmap. | Proposal review and faculty feedback incorporation. | **Project Proposal Document** |

| | | | | | |
|---|---|---|---|---|---|
| **August** | **Week 3** | Project Planning | Prepare Work Breakdown Structure (WBS), sprint plan, person-hour estimation, GitHub repository, Trello/Jira board, and Gantt chart. | Review project schedule and task allocation. | **Project Planning Document** |
| **August** | **Week 4** | Requirements Engineering | Prepare functional requirements, non-functional requirements, user stories, use cases, wireframes, interface requirements, and acceptance criteria. | Validate requirements and prepare preliminary test scenarios. | **Business Requirements Document (BRD)** |
| **August** | **Week 5** | System Design | Finalize software architecture, ER diagram, REST APIs, UML diagrams, database schema, UI mockups, and module design. | Prepare detailed unit test cases and integration strategy for all modules. | **System Design Document** |

| | | | | | |
|---|---|---|---|---|---|
| **September** | **Week 6** | **Module 1 – User Authentication & Authorization** | Develop registration, login, JWT authentication, profile management, and role-based access control. | **Unit Testing**, code review, bug fixing, and integration with the project framework. | Module 1 Completed |
| **September** | **Week 7** | **Module 2 – Workspace & Collaboration Management** | Develop workspace creation, participant management, invitations, and session management. | **Unit Testing**, **Integration Testing (Modules 1 & 2)**, bug fixing, and documentation update. | Module 2 Completed |
| **September** | **Week 8** | **Module 3 – Real-Time Collaborative Code Editor** | Integrate Monaco Editor, Socket.IO, collaborative editing, synchronization, and file management. | **Unit Testing**, **Integration Testing (Modules 1–3)**, collaborative workflow validation, and regression testing. | Module 3 Completed |
| **September** | **Week 9** | **Module 4 – Code Execution Engine** | Integrate the Piston API, execution console, language support, and execution history. | **Unit Testing**, **API Testing**, **Integration Testing (Modules 1–4)**, regression testing, and defect resolution. | Module 4 Completed |

| | | | | | |
|---|---|---|---|---|---|
| **October** | **Week 10** | **Module 5 – Activity Logging & Proof-of-Work Tracking** | Develop activity logging, contribution tracking, timestamp recording, and SHA-256 hash verification. | **Unit Testing**, **Integration Testing (Modules 1–5)**, verification of activity logs, regression testing, and documentation update. | Module 5 Completed |
| **October** | **Week 11** | **Mid-Semester Demonstration** | Demonstrate integrated Modules 1–5, collect feedback, and refine completed features. | **System Integration Review**, **Regression Testing**, bug fixing, and prototype validation before demonstration. | **Mid-Semester Demonstration** |
| **October** | **Week 12** | **Module 6 – Contribution Analytics & Report Generation** | Develop analytics dashboard, contribution summaries, report generation, and export functionality. | **Unit Testing**, **Integration Testing (Modules 1–6)**, report validation, and bug fixing. | Module 6 Completed |
| **October** | **Week 13** | **Module 7 – Dashboard & Project History** | Develop personalized dashboard, workspace history, project summaries, search, and filtering. | **System Testing**, **UI Testing**, **Integration Testing (Modules 1–7)**, and regression testing. | Module 7 Completed |

| | | | | | |
|---|---|---|---|---|---|
| **November** | **Week 14** | **Module 8 – Workspace Settings & Notifications** | Develop workspace settings, notification services, collaboration preferences, and finalize all functional features. | **End-to-End Testing**, **User Acceptance Testing (UAT)**, usability testing, and final feature integration. | Module 8 Completed |
| **November** | **Week 15** | **Quality Assurance & Release Preparation** | Resolve remaining issues, optimize application performance, prepare deployment packages, and finalize documentation. | **Performance Testing**, **Security Testing**, **Final Regression Testing**, database validation, and completion of **Test Plan & Test Cases**. | **Test Plan & Test Cases** |
| **November** | **Week 16** | **Deployment & Final Submission** | Deploy the application, prepare API documentation, README, user manual, final report, presentation, poster, and project video. | Deployment verification, smoke testing, and final release validation. | |

# Team Task Assignments

## Team Allocation Strategy

CodeTrail's development follows a **module-based method** of assigning tasks instead of adopting the conventional division between frontend and backend. With this approach, each development team is given **full responsibility for the functional modules** that have been assigned to it, covering the stages of requirement analysis, UI implementation, backend development, database integration, testing, documentation, and module integration.

The eight-member team is divided into two development teams:

### Team A (4 Members)
Manushri Thakkar
Ruchita Patadiya
Mahi Kansara
Monar

### Team B (4 Members)
Maryam Shaikh
Anistina Dsouza
Rishika Shah
Meet Sheth

At every stage of the planning, requirements engineering, system design, testing, deployment, and documentation, the entire team works together. In the implementation phase the development work is allocated according to the functional modules which have been assigned.

This method results in a **balanced allocation of work**, promotes **full ownership of features**, and makes it **easier to coordinate and integrate the project**.

## Common Team Responsibilities

The following activities are carried out collectively by all team members throughout the project lifecycle.

| Project Phase | Common Team Responsibilities |
|---|---|
| Project Initiation | Team formation, literature survey, technology evaluation, feasibility analysis, and project ideation. |
| Project Planning | Work Breakdown Structure (WBS), person-hour estimation, sprint planning, GitHub repository setup, Trello/Jira board creation, and Gantt chart preparation. |
| Requirements Engineering | Functional and non-functional requirements gathering, user stories, use cases, interface requirements, wireframes, and acceptance criteria. |
| System Design | Software architecture, ER diagram, REST API contracts, UML diagrams, database schema, and UI/UX design. |
| Quality Assurance | Continuous integration, regression testing, bug fixing, code reviews, performance optimization, and system validation. |
| Deployment & Documentation | Cloud deployment, API documentation, user manual, final report, presentation, project poster, and video walkthrough. |

## Module Allocation

| Team | Assigned Modules |
|---|---|
| **Team A**<br>Manushri Thakkar<br>Ruchita Patadiya<br>Mahi Kansara<br>Monar | **Module 1:** User Authentication & Authorization<br>**Module 3:** Real-Time Collaborative Code Editor<br>**Module 5:** Activity Logging & Proof-of-Work Tracking<br>**Module 7:** Dashboard & Project History |
| **Team B**<br>Maryam Shaikh<br>Anistina Dsouza<br>Rishika Shah<br>Meet Sheth | **Module 2:** Workspace & Collaboration Management<br>**Module 4:** Code Execution Engine<br>**Module 6:** Contribution Analytics & Report Generation<br>**Module 8:** Workspace Settings & Notifications |

## Team Member Responsibilities during Module Development

The responsibilities of each development team include the following activities:

- Carry out a **requirement analysis** by studying and analysing both the functional and non-functional requirements of the assigned module, determining the system dependencies, and drawing up implementation plans.

- For the **development of the user interface**: design and implement responsive and user-friendly interfaces using React.js and Tailwind CSS, making sure that they are consistent with the overall design of the application.

- For the **backend development**, create the server-side business logic, controllers, middleware, and application services using Node.js and Express.js in order to provide support for the required module functionality.

- When **designing and integrating databases**, we carry out the database operations using MongoDB and Mongoose, and link the persistent data storage with the application.

- To develop a **REST API**, design it, implement it, and provide the necessary documentation for use in communication between the frontend and backend.

- **Integrating third-party API services**: where appropriate, integrate services such that there is reliable communication between the application and the external platforms.

- When carrying out unit testing, it is necessary to **test individual components, APIs, and business logic** in order to ensure that each feature works correctly before they are integrated.

- For **integration testing**, it must be checked that all the interconnected components are able to communicate with each other and work correctly without causing any impact on the existing features.

- **Bug Identification and Resolution:** Continuously identify, track, and resolve defects discovered during development and testing to improve software quality and stability.

With this module-based method of development, each feature is designed, put into action, tested, examined, documented and then integrated before the team moves on to the next development iteration. The iterative process thus ensures continuous quality assurance, lowers the risks involved in integration, allows defects to be detected early on, and brings the project in line with standard industry Agile practices for software development.

# Work Breakdown Structure (WBS)

The Work Breakdown Structure (WBS) provides a hierarchical decomposition of the **CodeTrail** project into manageable phases, work packages, and development activities. It serves as the foundation for project scheduling, task allocation, resource planning, progress monitoring, and milestone tracking. Each work package represents a measurable project deliverable and is further divided into smaller tasks that can be assigned to the development teams.

## Work Breakdown Structure

### 1.0 CodeTrail – Real-Time Collaborative Code Editor with Contribution Tracking

#### 1.1 Project Initiation & Planning

- **1.1.1** Team Formation
- **1.1.2** Literature Survey
- **1.1.3** Technology Evaluation
- **1.1.4** Project Proposal Preparation
- **1.1.5** Project Planning
- **1.1.6** Work Breakdown Structure (WBS)
- **1.1.7** Project Timeline & Milestones
- **1.1.8** Jira Project Setup
- **1.1.9** GitHub Repository Setup

#### 1.2 Requirements Engineering

- **1.2.1** Functional Requirements
- **1.2.2** Non-Functional Requirements
- **1.2.3** User Stories
- **1.2.4** Use Case Specifications
- **1.2.5** Wireframes & UI Mockups (Initial)
- **1.2.6** Interface Requirements
- **1.2.7** Acceptance Criteria
- **1.2.8** Business Requirements Document (BRD)

#### 1.3 System Design

- **1.3.1** Software Architecture
- **1.3.2** Database Design (ER Diagram)

**1.3.3** REST API Design
**1.3.4** Class Diagram
**1.3.5** Sequence Diagram
**1.3.6** Component Diagram
**1.3.7** UI/UX Design (Final)
**1.3.8** System Design Document

## 1.4 Module Development

#### 1.4.1 Module 1 – User Authentication & Authorization

- User Registration
- User Login
- Profile Management
- Role-Based Access Control
- Session Management

#### 1.4.2 Module 2 – Workspace & Collaboration Management

- Create Workspace
- Join Workspace
- Participant Management
- Workspace Sessions
- Workspace History

#### 1.4.3 Module 3 – Real-Time Collaborative Code Editor

- Monaco Editor Integration
- Real-Time Synchronization
- Live Cursor Tracking
- File Management
- Auto Save & Recovery

#### 1.4.4 Module 4 – Code Execution Engine

- Language Selection
- Code Execution
- Output Console
- Execution History
- Execution Validation

#### 1.4.5 Module 5 – Activity Logging & Proof-of-Work Tracking

- Activity Logging
- Contribution Tracking
- Timestamp Recording
- SHA-256 Hash Verification
- Activity Validation

#### 1.4.6 Module 6 – Contribution Analytics & Report Generation

- Contribution Analytics
- Session Analytics
- Report Generation
- PDF Export
- Analytics Validation

#### 1.4.7 Module 7 – Dashboard & Project History

- User Dashboard
- Workspace History
- Activity History
- Search & Filter
- Dashboard Integration

#### 1.4.8 Module 8 – Workspace Settings & Notifications

- Workspace Settings
- Participant Permissions
- Real-Time Notifications
- Notification Preferences
- Module Validation

## 1.5 Continuous Integration & Quality Assurance

**1.5.1** Unit Testing
**1.5.2** Integration Testing
**1.5.3** Regression Testing
**1.5.4** System Testing
**1.5.5** Performance Testing
**1.5.6** Security Testing
**1.5.7** Bug Tracking & Resolution
**1.5.8** Test Documentation

## 1.6 Deployment & Release

**1.6.1** Application Deployment

**1.6.2** Environment Configuration
**1.6.3** Deployment Verification
**1.6.4** API Documentation
**1.6.5** User Manual
**1.6.6** README Preparation
**1.6.7** Deployment Notes

### 1.7 Project Documentation

**1.7.1** Weekly Progress Reports
**1.7.2** Mid-Semester Demonstration
**1.7.3** Test Plan & Test Cases
**1.7.4** Final Project Report
**1.7.5** Project Poster
**1.7.6** Project Video Walkthrough
**1.7.7** Final Presentation Slides

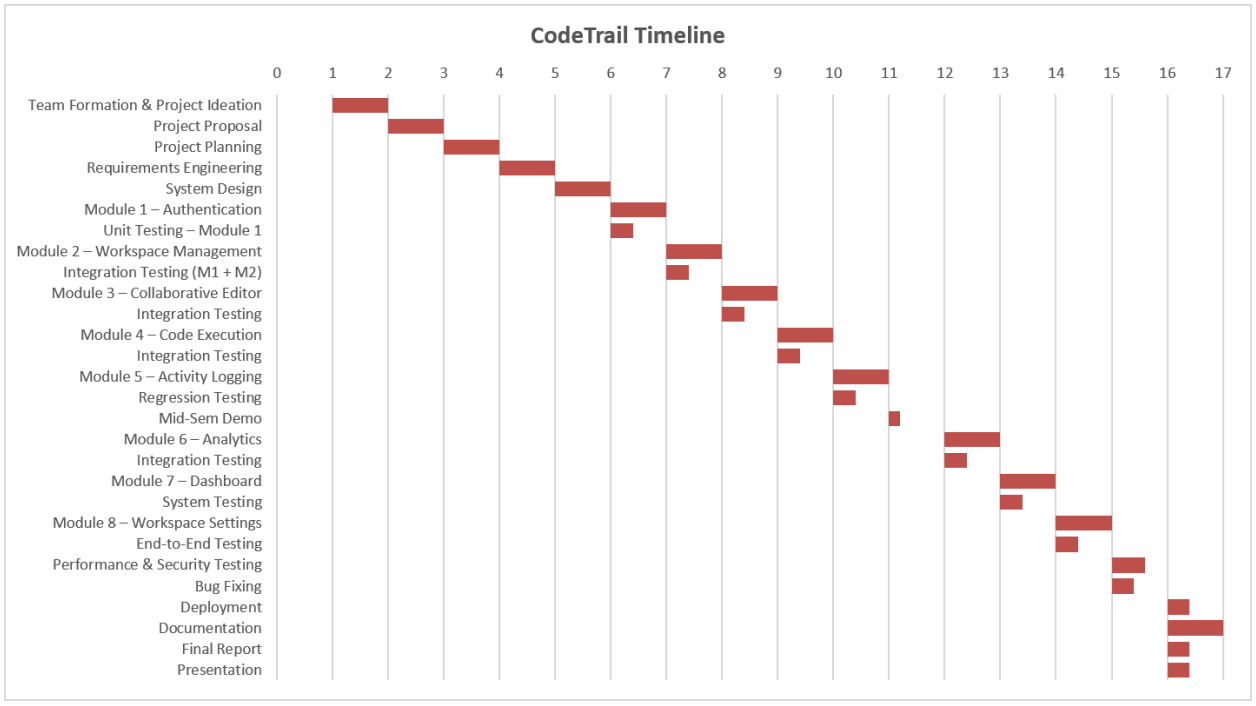
### 1.8 Project Closure

**1.8.1** Final Demonstration
**1.8.2** Peer Review
**1.8.3** Final Repository Submission
**1.8.4** Project Completion & Handover

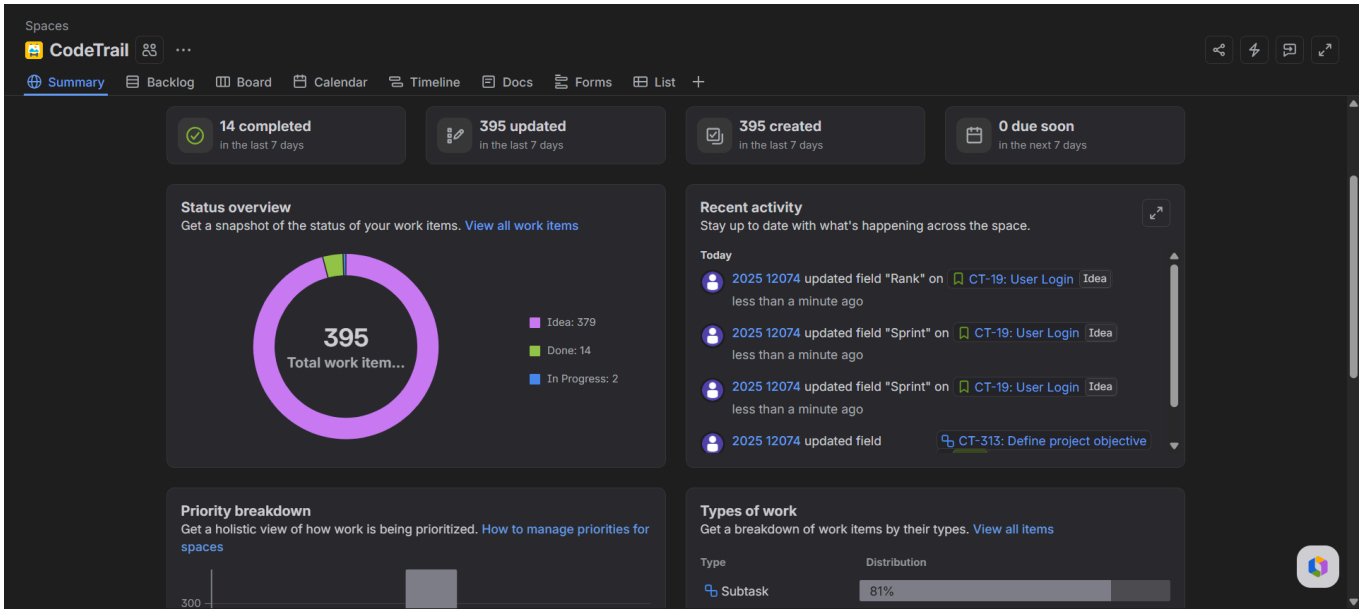
# Project Management Tools

| Tool | Purpose | Usage in the Project |
|---|---|---|
| **Jira** | Task & Sprint Management | Used to create Epics, Stories, and Subtasks, assign work to team members, monitor progress, manage sprints, and track the status of development activities. |
| **GanttProject** | Project Scheduling | Used to prepare the project timeline, define task dependencies, schedule milestones, and visualize the overall development plan using a Gantt chart. |
| **GitHub** | Version Control | Used for source code management, branch management, issue tracking, code reviews, and collaborative development. |

# Gantt Chart




# Jira Project Board



https://dau-team-vpcte9es.atlassian.net/jira/software/projects/CT/summary

## GitHub Repository

The source code, project documentation, Jira planning artifacts, Gantt chart, and all project deliverables are maintained in the project's GitHub repository.

https://github.com/Manushri3080/CodeTrail.git